

7.1.jpg

Sistemul presupune:

- Modelarea **partidelor, candidaților, alianțelor** și **schemelor de vot**.
- Gestionarea unor **alegeri** pentru diverse niveluri (local/comunal/județean/național).
- Introducerea manuală a voturilor și **clasament electoral** rezultat.

1. MODELUL DOMENIULUI

a. Diagrama de clase (conceptuală)

Vom avea următoarele clase principale:

- SchemaElectorală
- Candidat
- Partid
- Alianță
- Regiune (comună/oras/judet/tara)
- Vot (rezultatele pentru un candidat)

```

+-----+
| SchemaElectoral |
+-----+

| id          |
| denumire    |
| data        |
| tip         | <- local, judetean, etc.
| regiune: Regiune |
+-----+

| +getRezultate() |
+-----+

```



|

|

```

+-----+ +-----+ +-----+
| Candidat | ◀-----| Partid  | ◀-----▶ | Alianta  |
+-----+ +-----+ +-----+

| nume      | | nume      | | nume      |
| nrVoturi  | | membri: [] | | partide: [] |
+-----+ +-----+ +-----+

```

```

+-----+
| Regiune |
+-----+

| nume     |
| tip      | <- comuna/oras/judet/tara
+-----+

```

```

+-----+
|  Vot    |
+-----+
| candidat |
| nrVoturi |
+-----+

```

b. Diagrama de pachete (modularizare logică)

```

+-----+
|  model    | -> clasele de business logic (Partid, Candidat, SchemaElectorală, etc.)
+-----+

```

```

+-----+
|  data      | -> clase pentru DB access (DAO, Repositories)
+-----+

```

```

+-----+
|  service   | -> logica de business (ex: calcul procent, clasament)
+-----+

```

```

+-----+
|  main/ui   | -> cod de testare sau UI (dacă era necesar)
+-----+

```

c. Pattern-uri cerute:

Singleton (conexiune DB)

Se folosește pentru a avea o singură instanță a conexiunii la baza de date.

```

public class DatabaseConnection {
    private static DatabaseConnection instance;
    private Connection connection;
}

```

```

private DatabaseConnection() {
    // Initializează conexiunea JDBC
}

public static DatabaseConnection getInstance() {
    if (instance == null) {
        instance = new DatabaseConnection();
    }
    return instance;
}

public Connection getConnection() {
    return connection;
}
}

```

Composite (pentru ierarhia Partide → Aliante → Membri)

```

public interface ElementPolitic {
    String getNume();
    int getTotalVoturi();
}

public class Partid implements ElementPolitic {
    private String nume;
    private List<Candidat> membri;

    public int getTotalVoturi() {
        return membri.stream().mapToInt(Candidat::getNrVoturi).sum();
    }

    // restul codului...
}

public class Alianta implements ElementPolitic {
    private String nume;
    private List<Partid> partide;
}

```

```
public int getTotalVoturi() {  
    return partide.stream().mapToInt(Partid::getTotalVoturi).sum();  
}  
}
```

2011_06_eng_1

1. Three architectural styles

a) Pentru fiecare stil trebuie aplicat la:

- Un sistem de colectare de date web
- O funcție de alarmă într-un telefon

Exemplu de răspuns:

i. Layered Architecture

- **Descriere:** Sistemul este împărțit în straturi (UI, business logic, persistence).
- **Colectare date web:** Ideal – UI colectează, BL prelucrează, DB salvează.
- **Alarmă în telefon:** Poate fi prea greoi, dar dacă include notificări, istoric, setări → util.

ii. Client-Server

- **Descriere:** Separarea clientului (interfața) de server (logică, date).
- **Colectare date web:** Ideal – client trimite date, server stochează/statistici.
- **Alarmă telefon:** Nu prea potrivit – depinde de conexiune.

iii. Event-Driven

- **Descriere:** Componente care reacționează la evenimente.
- **Colectare date web:** Poate fi aplicat dacă fiecare trimitere de date e un eveniment.
- **Alarmă:** Foarte potrivit – răspunde la evenimentul “timp curent = alarmă”.

2. Card-based access control system – Quality Attributes

Scenarii de calitate:

- **Performance:** Acces rapid la poartă (cardul trebuie citit < 1s)
- **Reliability:** Sistemul funcționează chiar dacă baza de date e temporar offline
- **Security:** Cardurile sunt criptate, autentificarea este sigură

- **Usability:** Ușor pentru utilizatori (feedback vizual, sonor)

Tactici:

- Caching pentru performanță
- Failover mechanism pentru disponibilitate
- Encryption și autentificare pentru securitate

3. Mortgage system – Model + OCL constraints

Entități:

- Person (id, income)
- Asset (value, owner)
- Mortgage (value, monthlyPayment, startDate, endDate, asset)

OCL constraints:

-- o persoană poate ipoteca doar un asset pe care îl deține

context Mortgage

inv onlyOwner:

self.asset.owner = self.person

-- ipoteca se face înainte de data de sfârșit

inv validDate:

self.startDate < self.endDate

-- o persoană are un ID unic

context Person

inv uniqueId:

Person.allInstances()->isUnique(p | p.id)

-- suma tuturor ipotecilor < 50% din venitul persoanei

context Person

inv maxMortgage:

self.mortgages->collect(m | m.value)->sum() < self.income * 0.5

-- suma ipotecilor < valoarea asset-ului

context Asset

inv underAssetValue:

```
self.mortgages->collect(m | m.value)->sum() < self.value
```

4. Car vs ElectricCar – Violare principii OOP

```
class Car {  
    public void drive() { ... burn fuel ... }  
    public void fillGas() { ... }  
}  
  
class ElectricCar extends Car {  
    public void drive() { ... use battery ... }  
    public void fillGas() { throw new NotImplementedException(); }  
    public void replaceBattery() { ... }  
}
```

Probleme:

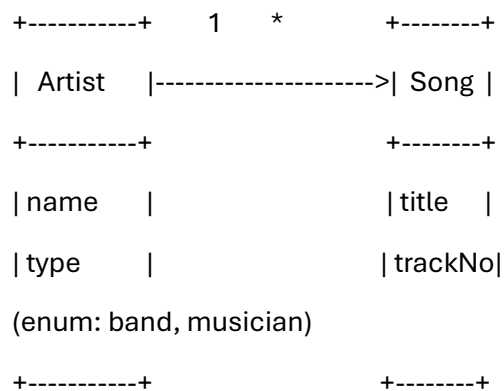
- Încalcă **Liskov Substitution Principle (LSP)**: o instanță de ElectricCar nu poate fi folosită în loc de Car, deoarece fillGas() aruncă excepție.
- Încalcă **Interface Segregation Principle (ISP)**: Car are metode care nu sunt relevante pentru toate subclasele.

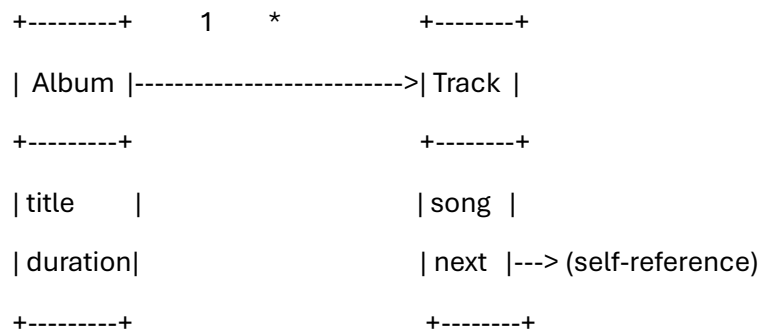
Soluție:

- Introdu o interfață FuelCar cu fillGas()
- Car devine clasă abstractă sau interfață generală cu drive()

5. Model artist, album, melodii

a. Class Diagram (simplificat):





Album -has-> multiple Tracks

Track -is-> one Song

Song -has-> one Artist

b. Design Patterns

1. **Strategy** – pentru diferite tipuri de artist (Band, Musician) – fiecare poate implementa o strategie de afișare/info.
2. **Composite** – un Band conține mai mulți Musicians, dar este tratat ca un Artist.

c. GRASP & Principles

- **Low Coupling** – Track știe doar de Song, nu de Album întreg.
 - **High Cohesion** – fiecare clasă are o singură responsabilitate.
 - **Creator** – Album creează Track-urile.
 - **Information Expert** – Artist deține date despre sine.
-

2011_summer_eng_1

1. Combinatorial Auction – Domain Model + 2 Design Patterns (fără Singleton)

Cerință:

Să modelăm o licitație în care utilizatorii pot licita pentru produse sau grupuri de produse. Avem:

- SimpleAuction (pentru un produs)
- CombinatorialAuction (pentru grupuri)
- Licitația are o logică de determinare a câștigătorului.

Design Patterns:

1. Strategy Pattern – pentru logica de determinare a câștigătorului


```

interface WinnerSelectionStrategy {
    Bid determineWinner(List<Bid> bids);
}

class SimpleWinnerStrategy implements WinnerSelectionStrategy { ... }

class CombinatorialWinnerStrategy implements WinnerSelectionStrategy { ... }

```

2. Composite Pattern – pentru Bundle de produse

```

abstract class ProductComponent {
    abstract double getPrice();
}

class Product extends ProductComponent {
    double price;

    double getPrice() { return price; }
}

class ProductBundle extends ProductComponent {
    List<ProductComponent> items;

    double getPrice() {
        return items.stream().mapToDouble(ProductComponent::getPrice).sum();
    }
}

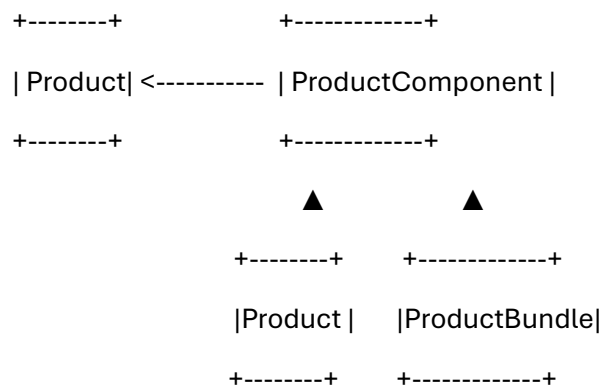
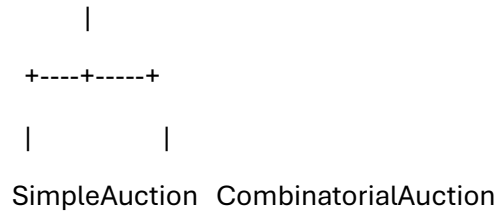
```

Model de Domeniu

```

+-----+ 1 * +-----+
| Auction |----->| Bid |
+-----+          +-----+
| id, type |          | amount |
| strategy |          | buyer |
+-----+          +-----+
|

```



2. Personal Calendar App – High-Level Architecture

Cerință:

- Calendar cu adăugare/ștergere programări
- Alarmă vizuală/audio dacă un eveniment e < 1h
- Arhitectură modulară

Componente:

- CalendarService: gestionează evenimentele
- AlarmChecker: verifică dacă un eveniment este iminent
- Notifier (interfață) – ConsoleNotifier, AudioNotifier
- UserInterface – primește comenzile utilizatorului

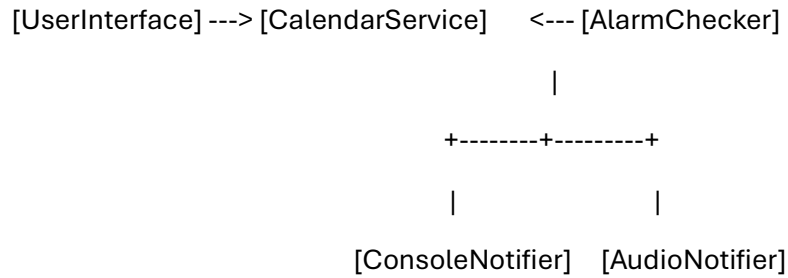
Patterns:

Observer Pattern – pentru alarme multiple

- AlarmChecker notifică toți observatorii (ConsoleNotifier, AudioNotifier)

Command Pattern – pentru comenzile userului: adăugare/ștergere

Arhitectura (textuală):



3. Stream Library – Decorator Pattern

Cerință:

- Librărie de streamuri (MemoryStream, FileStream)
- Se pot adăuga capabilități (compresie, criptare etc.) la un stream existent
- Fără diferență în codul client

Pattern folosit: Decorator

Interfețe și clase:

```
interface Stream {
    void write(String data);
    String read();
}

class FileStream implements Stream { ... }

class MemoryStream implements Stream { ... }

abstract class StreamDecorator implements Stream {
    protected Stream stream;

    StreamDecorator(Stream s) { this.stream = s; }
}

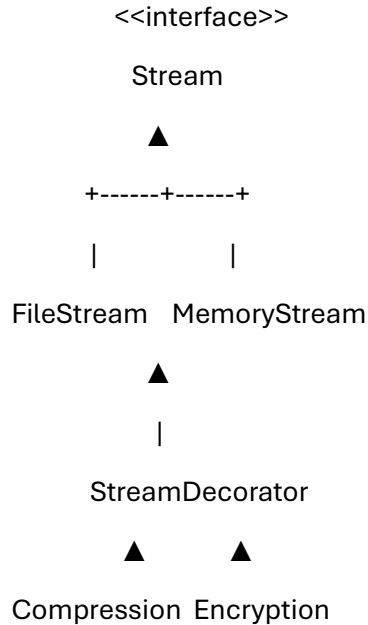
class CompressionStream extends StreamDecorator { ... }

class EncryptionStream extends StreamDecorator { ... }
```

Exemplu de utilizare:

```
Stream s = new CompressionStream(new EncryptionStream(new FileStream()));
s.write("data");
```

Class Diagram (textual):



Cum elimini dependențele ciclice între pachete?

Soluții:

1. **Inversarea dependenței (DIP)** – clasele depind de interfețe, nu de implementări.
2. **Observer/Event** – reduce nevoia de apeluri directe între clase.
3. **Mediator** – comunicarea centralizată.
4. **Refactorizare logică a pachetelor** – reorganizare în module independente.

Exemplu:

- UI → Service → UI → devine: UI → IService, Service implementează IService.

Ce este un Identity Map și cum îl proiectezi?

Definiție:

- **Identity Map** este un cache care stochează obiecte deja încărcate din DB
- Evită crearea de instanțe duplicate
- Asigură consistență și performanță

Opțiuni de design:

```

class IdentityMap {
    private Map<String, Object> cache;

    public Object getByld(String id) {

```

```
        return cache.get(id);
    }

    public void put(String id, Object obj) {
        cache.put(id, obj);
    }
}
```

Se folosește de obicei în:

- ORM frameworks
 - Unit of Work Pattern
-

2012_ro_1

Cerință generală

Sistemul de gestionare a creditelor unei instituții financiare, cu funcționalități precum:

- administrare clienți
- administrare credite și conturi curente
- procesare plăți automate
- notificări automate către angajați

1. Trei atribute de calitate + tactici

a) Fiabilitate (Reliability)

- **Descriere:** Sistemul trebuie să funcționeze corect în fiecare zi la ora 8 (verificarea ratelor).
- **Tactică:**
 - Redundanță (backup servere)
 - Heartbeat pentru servicii automate

b) Securitate (Security)

- **Descriere:** Acces pe bază de user/parolă, date financiare sensibile.
- **Tactică:**
 - Autentificare + autorizare (RBAC)
 - Criptarea comunicării și a bazei de date

c) Extensibilitate

- **Descriere:** Adăugare de noi tipuri de clienți (platinum, gold).

- **Tactică:**
 - Utilizarea unor ierarhii polimorfe pentru ClientType
 - Pattern Strategy/Factory pentru comportamente
-

2. Analiza arhitecturilor

Arhitecturi comparate:

1. **Layered Architecture**
2. **Service-Oriented Architecture (SOA)**
3. **Microservices**

a) Extensibilitate

- **Layered:** bună – UI/BL/DB separate
- **SOA:** foarte bună – noi servicii adăugate fără impact major
- **Microservices:** excelentă – fiecare serviciu e independent

b) Performanță (latență)

- **Layered:** decentă
- **SOA:** posibil overhead din cauza rețelei
- **Microservices:** pot apărea întârzieri la multe API calls

c) Integrare cu alte sisteme

- **Layered:** slabă – monolit greu de conectat
- **SOA:** excelentă – API bine definite
- **Microservices:** excelentă – RESTful/async messaging

Recomandare: SOA oferă un bun echilibru între modularitate și performanță

3. Model de domeniu (diagramă de clase)

Clase:

- Client (nume, tip, listă credite, cont curent)
- Credit (sumă, dobândă, rată, scadențar)
- Scadentar (dată, sumă, plătit: boolean)
- ContCurent (sold)
- Plată (sumă, dată, credit)

- Angajat
- Notificare

Client "1" ----- "*" Credit

Client "1" ----- "1" ContCurent

Credit "1" ----- "1..*" Scadentar

Credit "1" ----- "*" Plata

Angajat "*" <-----> "*" Notificare

4. Constrângeri OCL

-- un credit are scadențar cu minim o rată

context Credit

inv mustHaveInstallments:

self.scadentar->size() >= 1

-- rata e considerată plătită doar dacă există o plată aferentă

context Scadentar

inv checkIfPaid:

self.platit = self.credit.plati->exists(p | p.data = self.data)

-- dacă nu există fonduri, nu se face debitarea

context Plata

inv enoughBalance:

self.client.contCurent.sold >= self.suma

5. Design Patterns (exclus Singleton)

a) Strategy Pattern – pentru calculul scadențarului

- Creditul poate avea tipuri diferite (ipotecar, consum etc.)
- Fiecare tip are un algoritm diferit de generare a ratelor

interface ScadentarStrategy {

List<Scadentar> genereaza(double suma, double dobanda, int luni);

}

class CreditIpotecarStrategy implements ScadentarStrategy { ... }

b) Observer Pattern – pentru notificări automate

- Sistemul notifică angajații conectați dacă plata automată eșuează

```
interface Observer {
```

```
    void notifica(Client client, String mesaj);
```

```
}
```

```
class AngajatConectat implements Observer { ... }
```

Diagramă de clase

Include:

- Credit are o referință la ScadentarStrategy
 - Notificator abstract + implementări pentru angajați conectați
-

2014_06_eng_1

1. Decorator vs Chain of Responsibility – Similarități și Diferențe

Decorator Pattern

- **Scop:** Aduge responsabilități **dinamice** unui obiect.
- **Structură:** Obiectele se înfășoară unul pe altul.
- **Exemplu:** Stream I/O (e.g., BufferedInputStream peste FileInputStream în Java)

```
InputStream input = new BufferedInputStream(new GZIPInputStream(new
FileInputStream("file.txt")));
```

Chain of Responsibility

- **Scop:** Delegă cereri într-un **lanț de handleri** până când unul o procesează.
- **Structură:** Fiecare handler are o referință către următorul.
- **Exemplu:** Lanț de procesare a cererilor într-o aplicație web (interceptori, middleware)

```
abstract class Handler {
```

```
    Handler next;
```

```
    abstract void handle(Request req);
```

```
}
```

Similarități:

- Ambele sunt bazate pe **compoziție** și permit modificări comportamentale la runtime.
- Flexibile și deschise extensiei (Open/Closed Principle)

Diferențe:

Decorator

Adaugă funcționalități

Toate decorațiile sunt aplicate

Focalizat pe *comportament cumulat*

Chain of Responsibility

Delegă responsabilitatea

Doar unul dintre handleri răspunde

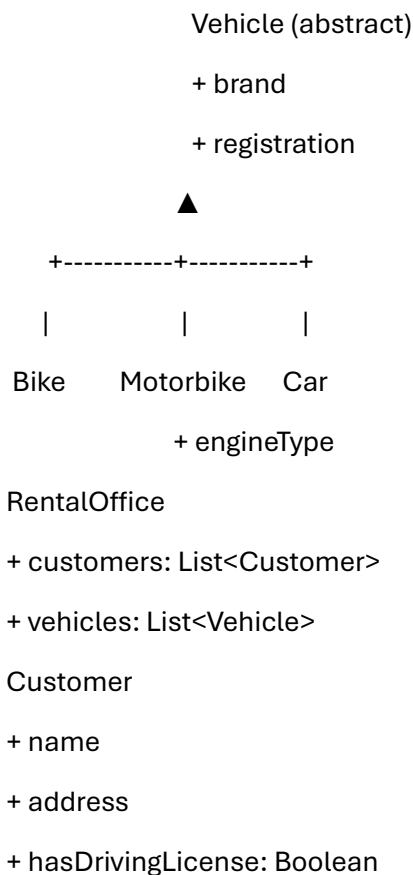
Focalizat pe *procesare selectivă*

2. Car Rental Office – UML + OCL

Domeniu:

- Vehicule: Bike, Motorbike, Car
- Vehicle (abstract), are: brand, registration, engineType (pentru car/motorbike)
- Customer: nume, adresă, permis de conducere
- RentalOffice: listă clienți, listă vehicule

UML Class Diagram



OCL Constraints (2 exemple):

1. Un client fără permis nu poate închiria Bike:

context Rental

inv cannotRentBikeWithoutLicense:

self.vehicle.ocllsTypeOf(Bike) implies self.customer.hasDrivingLicense

2. Un client nu poate închiria mai multe vehicule simultan:

context Customer

inv singleRental:

self.rentals->size() <= 1

3. Social Network – Design Patterns + UML

Cerințe:

- Grupuri cu membri, tipuri (open/closed), wall page
- Scrierea în grup → vizualizare pentru toți membrii

Design Patterns:

a) Observer Pattern

- Membrii unui grup sunt notificatori pentru wall updates.

b) Composite Pattern

- Grupurile pot conține alte grupuri („related groups”).

UML (textual):

Group

+ title

+ type (open/closed)

+ wall: Wall

+ members: List<User>

+ relatedGroups: List<Group>

Wall

+ posts: List<Post>

Post

+ content

+ author

User

+ name

+ feed: List<Post>

Group are o listă de membri (User), care observă Wall-ul grupului.

4. Architectural Patterns: Active Record, Table Module, Data Mapper

Pattern	Descriere	Avantaje	Dezavantaje
Active Record	Fiecare entitate mapată direct la tabel, metode CRUD în clasă	Simplu, rapid	Violare SRP, greu de testat
Table Module	O clasă gestionează un tabel întreg, metodele sunt statice	Separă logică de date	Mai puțin OOP
Data Mapper	Entitățile sunt simple, iar mapperul face legătura cu DB	Decuplare totală, testabil	Mai complex de implementat

Din perspectiva principiilor SOLID/GRASP:

- **Active Record** = mai slab pe SRP, dar ok pentru aplicații mici
 - **Data Mapper** = mai potrivit pentru sisteme mari/modulare
 - **Table Module** = potrivit pentru scripturi, mai puțin OOP-centric
-

1. Asemănări și diferențe între State și Strategy Pattern

Aspect	State Pattern	Strategy Pattern
Scop	Comportamentul se schimbă în funcție de stare	Comportamentul este ales dinamic
Exemplu	Automat de cafea (stare: „fără bani”, „plătit”)	Algoritm de sortare diferit la runtime
Legare	Obiectul își schimbă starea	Obiectul delegă comportamentul
Context	Obiectul conține instanțe de stări	Obiectul conține instanță de strategie

Asemănări: ambele folosesc **compoziție**, aplică **Open/Closed Principle**

2. Principiul Liskov (LSP)

"Subclasele trebuie să poată fi folosite în locul superclaselor fără a modifica corectitudinea programului."

```
class Bird {  
    void fly();  
}  
  
class Ostrich extends Bird {  
    void fly() { throw new Exception(); } // încalcă LSP}
```

Soluție: refactorizare prin segregare:

```
interface Flyable { void fly(); }  
  
class Bird {}  
  
class Eagle extends Bird implements Flyable { ... }
```

Partea II

1. Use Case-uri (text-based)

Exemplu: pentru un sistem de gestiune a programărilor medicale

Use Case	Actor	Scop
Creează programare	Client	Salvează programare nouă
Notifică programare	Sistem	Trimite notificare automată
Anulează programare	Client	Șterge o programare existentă

2. Arhitectură conceptuală + architectural pattern

Pattern recomandat: Layered Architecture

Layer	Componente tipice
UI	Ecran rezervări, formular notificare
Service	ProgramareService, NotificareService
Data Access	ProgramareDAO, ClientRepository

Poze SD

1. Car Wash System – Design Pattern și Extensibilitate

Cerință:

- Spălătorie auto cu pachete (Bronze, Silver, Gold)
- Niveluri de curățare pe componente (roți + caroserie)
- Posibilitate de combinare: ex. Silver = Basic roți + Executive caroserie

Soluție: Strategy + Composite Pattern

Clase:

```
+-----+
| WashingProgram | <--- context
+-----+
| execute()      |
+-----+

+-----+
| CleaningStrategy | <--- Strategy
+-----+
| clean()         |
+-----+
```

+-----+	+-----+
BasicWheelCleaning	ExecutiveBodyCleaning
+-----+	+-----+
clean()	clean()
+-----+	+-----+

Combinare a strategiilor:

```
class BronzeWash extends WashingProgram {
    execute() {
        wheel.clean(); // Basic
        body.clean(); // Basic
    }
}
```

Extensibilitate:

- Adăugare de **noi tipuri de spălări**: Gold, Deluxe → doar combinăm alte strategii
- Adăugare de **noi componente**: ex. spălare plafon → se adaugă o nouă strategie

2. Class Management System – Layered Architecture

Cerință:

- Instructorul poate:
 - vedea clasele din ultimii 10 semestre
 - accesa lista studenților
 - seta note (dacă catalogul nu e închis)
- Notificare prin email la schimbarea notei

Soluție: 3-layer architecture + Observer Pattern

Straturi:

1. Presentation (UI)

- WebApp (React, Angular etc.)
- Afișează clase, studenți, note

2. Business Logic (Service)

- GradeService, NotificationService
- Validare: catalog deschis

3. Data Access Layer (DAL)

- StudentDAO, GradeDAO

Avantaje:

- Separare clară a responsabilităților
- Scalabilitate (poți înlocui părți ușor)

Dezavantaje:

- Comunicare între layere poate introduce latență
- Necesită mai multă infrastructură/testare

3. Package Design Principles – Evaluare + Corectare

Analiză a diagramei oferite: Posibile probleme:

1. Circular Dependency:

- Dacă Change → UML 2.1 MetaModel și Parser → Change, avem lanțuri periculoase.

2. Violare DIP:

- Comparator sau TestContainer ar putea depinde de implementări concrete.

Recomandare:

- Introduceți **interfețe** pentru ca modulele să depindă de **abstracții**
- Folosiți **facade** sau **mediator** pentru a gestiona accesul comun
- **Refactorizați în layere**: Parser + Comparator într-un pachet de analiză

4. Health Monitoring System – Arhitectură și Pattern-uri

Cerință:

- Monitorizare temperatură/puls/tensiune
- Istoric salvat (ora/zi)
- Vizualizare + statistici
- Funcționează pe dispozitiv mobil

Arhitectură: Model-View-Controller (MVC) + Observer + Repository

Componente:

View (Mobile UI)



Controller (User actions)



Model (Readings, Statistics)



Repository (DB or Cloud Sync)

Pattern-uri folosite:

- **MVC:** pentru separarea logicii aplicației
 - **Observer:** UI se actualizează când apar noi citiri
 - **Strategy:** pentru diferite tipuri de statistică (zilnic, orar, medie etc.)
-